What Consolidation Destroys: Measuring Information Loss in LLM-Maintained Hierarchical User Memory

Anonymous Author(s)

Abstract

Hierarchical memory systems for LLM agents periodically consolidate their own contents: sibling notes are merged, shared traits are promoted to a parent, stale nodes are pruned. Consolidation is normally justified by footprint and evaluated by downstream accuracy, so what it removes is never observed directly. We deploy such a memory as the persistent user model of a large Korean web map service and measure the removal itself.

We track every keyword in a child note across one consolidation pass and classify it as *retained*, *lifted* to the parent, or *lost* entirely. On 500 production memory trees, unguarded trait elevation with gemma-4-31b loses 25.3% of child keywords — counting only those absent from the surrounding prose as well — and loses them even where siblings share no keyword at all and nothing needs to move. That stratum still loses 18.0%: the deletion looks collateral to the rewrite rather than a consequence of compression.

Three findings follow. First, the magnitude is a property of the backbone, not of the method: across three models the same prompt ranges from 34% keyword compression to 5% *inflation*, and the prompt-level guard that is decisive on one model slightly but significantly *increases* loss on the model we actually deployed. Second, that guard protects the wrong tokens — place and brand names survive above their base rate, while what is lost in bulk is category vocabulary and, at 2.4× its base rate, the behavioural evidence grounding each claim; a guard rewritten from the measured loss profile cuts loss from 0.181 to 0.160 ($p < 0.0001$). Third, sibling merging, which prior work leaves unexamined, turns out to relocate rather than delete: it appears to contribute roughly a quarter of total loss under a same-path instrument and +0.007 under one that follows relocation.

The loss is not recovered downstream: over 70% of lost keywords are also absent from the regenerated persona, and not because a different subtree was selected. On a leak-free held-out MCQ, accuracy on the development half orders inversely to loss (0.452 to 0.484) but no pairwise difference is significant at N=500, so we report consolidation as a compression mechanism whose information cost is now measurable and, on one backbone, reducible, not as an accuracy lever. Every loss result was developed on one half of each snapshot and confirmed on a reserved half opened once (the instrument was corrected afterwards and both halves recomputed): the aggregate loss measurements replicate to within 0.003, and the accuracy null replicates too.

CCS Concepts

• **Information systems** → **Recommender systems**; • **Computing methodologies** → *Natural language processing*; *Knowledge representation and reasoning*.

WWW '27, Dublin, Ireland
2027. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Keywords

user modeling, personalization, memory-augmented LLM agents, memory consolidation, information loss

1 Introduction

Web services increasingly hand the user model behind their personalization to LLM agents that read it and keep it current; this paper measures what that upkeep destroys, in a deployed map and place-search service (2.0M distinct users over five weeks). In POI (Point-of-Interest)-centric services the model must capture which place types a user prefers and how they visit. Real POI logs make that hard: they are large-scale and noisy, explicit signals such as reviews are sparse, and implicit ones are pervasive but ambiguous [24]. Neither of the standard constructions fits. Full-context prompting hits context limits and lost-in-the-middle effects [19]: strong on factual recall (92.85% on LoCoMo [18]) but behind a compact agentic memory on implicit personalization (45.6% for GPT-5-Chat vs. 55.2% [12]), at much higher cost. Retrieval over flat chunks [29, 30] avoids the limit but cannot move between fine-grained categories and higher-level tendencies such as a dining-focused lifestyle. Our own measurements narrow this motivation: at the size our deployed memory reaches, flattening it away costs nothing downstream and selecting from it costs a great deal (Section 4.8); the hierarchy buys bounded read cost and a place for a maintenance policy, not accuracy.

We deploy a POI-centric Hierarchical Note-based Memory and a Memory Agent that maintains it (Figure 5 in Appendix A gives a worked example). The memory aggregates POI activities into topic-specific notes via LLM reasoning and organizes them in a three-level POI taxonomy.

Every such system must eventually consolidate, or its trees grow without bound: on our deployment, where consolidation has effectively not run, notes per user rise 16.4% and keywords 29.5% per update cycle. What consolidation costs to stop that has not been measured. Prior systems justify it by footprint and validate it by downstream accuracy [12, 14], which leaves the operation unobserved: a consolidation that quietly discards half the evidence and one that losslessly reorganizes it are indistinguishable if both shrink the tree and neither moves an accuracy number carrying its own noise.

We instrument the operation rather than its consequences, following each keyword through one consolidation pass on deployed memory. The resulting picture is not the one the design assumed, and not the same on every backbone.

Our contributions are:

- **A direct measure of consolidation loss.** Every child keyword is classified as retained, lifted, or lost across one pass: the operation is observed, not inferred from downstream scores, and needs no labels.

- **Evidence that consolidation deletes rather than compresses.** Users whose siblings share no keyword — where promotion has nothing to promote — still lose nearly a fifth of their child keywords, and loss rises monotonically with the amount genuinely shared.
- **Backbone dependence, with a sign change.** The same prompt on the same trees ranges from heavy compression to net inflation across three models, and the guard decisive on one significantly increases loss on another — the one we deployed.
- **A guard derived from measurement, and a mis-attributed source of loss.** The deployed guard names tokens that were never at risk; rewriting it around the measured loss profile reduces loss further at no additional inference cost. Merging, which prior work ignores, turns out to relocate rather than delete; a same-path instrument reports it as roughly a quarter of all loss.
- **Propagation to the consumed artifact, and a larger loss beside it.** Over 70% of the keywords consolidation removes also vanish from the regenerated persona and not because a different subtree was selected. Measuring the delivery path itself against the raw notes then found it the more expensive operation, by roughly a factor of six: selection and synthesis together cost +0.190 accuracy, three quarters of the gain the deployed memory delivers over an empty persona.
- **Confirmation on reserved data.** Each snapshot was split in half; every loss comparison was developed on one half and verified on the other (with one post-hoc correction, Section 4.2), including the guard derived from the development half.

2 Related Work

Memory architectures and their maintenance operations. Building on LLM-agent frameworks [32–34], agent memory varies in structure (flat vs. hierarchical) and in maintenance (append-only vs. actively managed; cf. [6]). One family rewrites or discards what it holds. Mem0 [14] issues per-item update and delete decisions over a flat list; MemGPT [17] pages content out of context; MemoryBank [43] applies Ebbinghaus-curve forgetting; PersonaMem-v2 [12] trains a 4B agent to rewrite a single compact persona memory as conversations arrive, and beats GPT-5-Chat (55.2% vs. 45.6%) at 16× fewer tokens than the full history. The other family does not contract: A-Mem [15] creates, links and updates notes with no described merge or prune step, EMem [16] stores information in a non-compressive event graph and exceeds full-context quality on LoCoMo with ~32× fewer tokens, and HippoRAG [44] indexes through a knowledge graph. PersonaMem [13] benchmarks dynamic user profiling. Our deployment belongs to the first family, with dynamic routing, trait elevation and a third extraction axis — and this paper is about what its rewriting step removes.

The measurement gap. Contraction is justified by footprint and validated by downstream task scores, a pairing that cannot distinguish a step that reorganizes losslessly from one that discards evidence: both shrink the store, and a noisy task score will not separate them. We know of no benchmark in this line that reports what a maintenance pass removed, only what the memory scored afterwards, so we instrument the operation itself.

Faithfulness under abstraction. Trait elevation is abstractive summarization applied to a system's own state, and abstractive summarizers are known to hallucinate and distort source content [1, 2]. The memory setting is harsher: a summarizer leaves its source intact, so a failure is recoverable by re-reading, whereas a memory agent overwrites, so what one pass discards is absent from every later read, and there is no source to check against at maintenance time — which is why the loss goes unnoticed. Input-side prompt compression [3] maps the same frontier for transient context; ours is persistent state, where each pass compresses an already-compressed artifact.

Prompt sensitivity and backbone dependence. LLM behaviour shifts under prompt changes a human would call cosmetic [4], motivating evaluation across multiple prompts [5]. That literature measures how a prompt's *score* moves; we report the same fragility in what a prompt *destroys*, across backbones rather than formats: with the prompt and the trees held fixed, one model compresses by a third while another inflates, and a guard that is decisive on the first significantly increases loss on the second.

Evaluating personalization without online tests. Classical sequential recommenders [37, 38] and trajectory models [39, 47, 48] learn dense representations with limited interpretability; LLM-based recommenders [40, 41, 49, 52, 54] unify recommendation as language processing. Persona generation has been studied from synthetic seeds [20], reward signals [23] and interaction histories [24], with parameter-side personalization [46, 53] and retrieval-based persona selection [26, 27] as neighbouring routes; for POI, zero-shot next-POI ranking is order-sensitive and weak on geography [25]. Evaluation methodology comes from PersonaGym [21], PersonaBench [22] and LaMP [45], with MCQ probing following MMLU [42]; where online tests are impractical, LLM judges are the emerging middle ground [7–9].

Full context as a reference point. Long-context LLMs lead on factual recall [18] but degrade on middle-position recall [19], needle-in-a-haystack tests [55] and RULER [51]; RAG [35, 36] sidesteps the limit but depends on retrieved passages containing the evidence needed to answer. Full context over a user's *raw* interaction log is 73K tokens and not an option — that is why a memory exists. Full context over the *memory* is a median 2,146 characters and entirely an option, which is why we use it as a baseline in Section 4.8 rather than as a strawman.

3 Proposed Method

3.1 Overview

Our framework constructs POI-centric user memory in three stages. **(1) Signal Extraction** converts raw POI interaction data (clicks with their originating search query, saves, reviews) into structured profiles, processing explicit signals (reviews) and implicit signals (behavioral logs) separately. **(2) Hierarchical Note Memory** stores extracted features in a taxonomy-grounded three-level tree, maintained by a **Memory Agent** that performs forward aggregation of new signals and backward consolidation for structural integrity. **(3) Downstream Consumption** selects relevant paths from the

hierarchy and synthesizes their notes into a short persona, which is injected into an LLM-as-reranker prompt [50] as the production target. Both of its steps are themselves compressions, and Section 4.8 finds them the most expensive operations we measure — more expensive than the consolidation this paper set out to instrument.

3.2 POI Interaction Signal Extraction

Two channels feed the memory. **Explicit** signals come from reviews, where a preference is stated outright; an LLM turns each review history into structured items with a claim and a quoted rationale. **Implicit** signals come from behaviour — POI clicks with the query that produced them, and saves — grouped into micro-sessions when consecutive interactions share a query or POI within 60 minutes. The sessionized log is profiled by an LLM under constraints that every claim cite specific click evidence and that unobserved behaviour never be inferred. Extraction runs over sliding 21-day windows whose results are merged. Appendix E gives the prompts and per-user variants.

3.3 Hierarchical Note Memory Structure

The core data structure is a three-level tree grounded in a predefined POI taxonomy.

Level 1 is the root, representing the user as a whole; Level 2 holds 10 broad POI domains (*Dining*, *Café/Dessert*, *Transportation/Mobility*, *Travel/Accommodation*, *Leisure/Sports*, *Culture/Arts*, *Self-development*, *Pets*, *Shopping*, *Beauty*); Level 3 holds 2–10 fine-grained subcategories per domain. A *Medical* domain was specified initially and removed before deployment as sensitive personal information; Section 5 describes the filtering that replaced it. Unlike a fixed-slot taxonomy, the agent may create new Level-3 nodes when user data warrants categories the base hierarchy lacks, subject to the consistency constraints of Section 3.4. Depth is bounded at three levels by prompt instruction only; we found no structural check in the deployed code, and Section 5 reports whether the bound held in our runs.

Each node carries a *Note*: its `path`, a natural-language `summary` of the user's preferences for that category, BM25-oriented `keywords` for retrieval, and update bookkeeping (`updated_count`, `recently_updated`). The keyword field is what our instrument tracks. A `user_info` field carried demographic attributes at the root; it was removed from the deployed schema during the course of this work under a privacy review (Section 5). Appendix B gives the full schema with examples and the storage layout.

Figure 6 walks one update through both passes: the forward pass touches only the notes new activity affects, and the backward pass then merges overlapping siblings, elevates traits shared across them to the parent, and prunes leaves untouched within a staleness window.

3.4 Memory Agent: Forward Aggregation

New interactions are folded in by three LLM steps: extract a preference profile split three ways (demographics, explicit, implicit); map each item onto an existing or newly created taxonomy path, preferring reuse and refusing paths deeper than three levels; and rewrite the note at each target path as an editor rather than an appender, resolving conflicts in favour of recency. For a user with N distinct target paths the cost is

$$\#\text{LLM calls}(u) \;=\; 2 + N(u), \tag{1}$$

one call for extraction, one for mapping, and one per path. Instrumenting the production log gives $N \approx 8.0$. Appendix F states the routing rules. Nothing here measures the forward pass; it produces the trees the next section consolidates.

3.5 Memory Agent: Backward Consolidation

Repeated forward passes introduce redundancy across siblings, scatter shared preferences across leaves, and leave stale nodes from transient interests. Backward consolidation addresses these in a fixed order: merging reduces the number of siblings, which yields clearer common traits for elevation, and pruning last removes nodes stale after reorganization.

3.5.1 Sibling Node Merging. For each parent with two or more children, an LLM is shown all child notes and asked which leaf siblings overlap semantically enough to merge. An approved merge yields a unified `merged_note` integrating the sources without duplication and inheriting the *maximum* `updated_count` among them (Section 5 explains why the maximum and not the sum); source nodes are then deleted. When only part of a source fits, the LLM returns an `updated_source` with the remainder, written back to the original node.

3.5.2 Common Trait Elevation. After merging, an LLM examines each parent–children group for preferences appearing across two or more siblings, rewrites the parent to incorporate them, and rewrites each child to remove the elevated content. This enforces a hierarchical invariant: shared preferences sit at the appropriate abstraction level rather than duplicated across subcategories.

The over-abstraction failure mode. Asked what "Korean," "Chinese," and "Japanese" notes share, the LLM replaces the concrete words with an abstract superclass — writing "*enjoys Asian cuisines*" at the Dining parent and stripping the concrete forms from each child. The hierarchy keeps its shape, but the persona loses the words a reranker needs to tell a Korean-leaning user from a Japanese-leaning one. Section 4 measures how often this happens and, more importantly, that it also happens where there is nothing shared to elevate.

Concrete-noun guard. We address this with a prompt-level rule that leaves the structural operation unchanged. The deployed elevation prompt carries one extra instruction: *proper nouns — place names, store names, brands — must be kept verbatim at the child and never lifted to the parent.* We compare this guard against two alternatives: the *destructive* variant (children rewritten to remove the elevated trait, no guard) and the *additive* variant (children left intact, accepting duplication). Section 4.5 shows that the deployed wording protects tokens that were never at risk, and derives a replacement from the measured loss profile.

Formally, let $C(n)$ denote the set of traits stored at note n. Given a parent node p with sibling set $\text{ch}(p)$, let $I = \{\, i : |\{n_k \in \text{ch}(p) : i \in C(n_k)\}| \geq 2 \,\}$ be the set of traits appearing in two or more siblings.

The elevation operation rewrites the parent and child contents as

$$C(p) \;\to\; C(p) \cup I, \qquad C(n_k) \;\to\; C(n_k) \setminus I, \quad \forall\, n_k \in \mathrm{ch}(p). \tag{2}$$

Let $S = \sum_n |C(n)|$ denote total stored content, and let $k_i = |\{n_k \in \mathrm{ch}(p) : i \in C(n_k)\}|$ denote how many siblings contain trait i (with $k_i \geq 2$ by definition of I). Assuming the elevated traits are not already at the parent ($I \cap C(p) = \emptyset$), the elevation reduces total content by

$$\Delta S \;=\; -\sum_{i \in I} (k_i - 1) \;\leq\; -|I|, \tag{3}$$

a hierarchical compression analogous to the Minimum Description Length principle [31]. To our knowledge, this explicit trait elevation operation has not been proposed in prior memory systems [12, 14–16].

3.5.3 Stale Leaf Pruning. Finally, a leaf is removed when it has been updated at most $\tau_u = 2$ times and last touched more than $\tau_d = 120$ days ago — transient interests that never became sustained preferences. Appendix C gives the rule and the traversal order.

4 Measuring Consolidation Loss

4.1 Instrument

Consolidation is normally evaluated by what survives in aggregate — tree size, or a downstream accuracy score. Neither says where a particular piece of evidence went, so we instrument the operation directly. Each note carries a `keywords` field: the concrete tokens extraction lifted from the user's activity (dish names, venue types, place names, behavioural evidence such as "repeated clicks"). These are the units a downstream reranker can act on, and being explicit rather than buried in prose they can be followed exactly. For every keyword at a non-root note before a consolidation pass, exactly one of three things is true afterwards (Figure 1):

- **retained** — still on the same note;
- **lifted** — gone from the note, now on its parent (the intended behaviour of elevation);
- **lost** — on neither, and absent from the prose of both, and absent from any note the pass *created* under the same parent.

Two clauses do the work, both from an audit of our own earlier measurements. The prose clause: a keyword can leave the `keywords` field while the concept survives in the summary beside it, and 19.2% of keywords missing from both keyword fields do still appear verbatim in the surrounding prose, so requiring absence from the text makes LossRate conservative on that axis rather than a keyword-churn count. The relocation clause: merging writes a unified note at a new path, so a rule inspecting only the original path and its parent scores relocated content as deleted — which inflates merging's apparent contribution by an order of magnitude (Section 4.6). We therefore scope survival to the parent, the original note, and paths the pass *created* beneath that parent: provably inert wherever nothing moved, and verified to be where merges land, since every new path in our full-backward runs appeared under a parent that had lost a child (340/340 and 338/338).

We do *not* credit survival by substring (*jiu-jitsu* inside *Brazilian jiu-jitsu*), which would apply to 20.9% of losses. Korean morphology

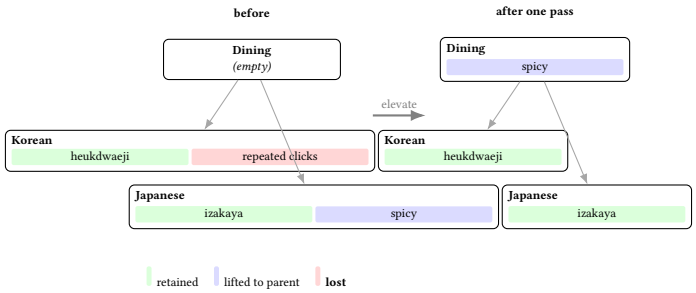


Figure 1: The instrument. Every keyword at a non-root note lands in exactly one bucket after a pass: still on its own note (*retained*), moved to the parent (*lifted*, the intended behaviour), or on neither (*lost*). Here "spicy" is correctly promoted and "heukdwaeji" kept, but "repeated clicks" — the evidence supporting a claim — is deleted without appearing anywhere.

cuts the other way, but normalizing josa and verb endings on both sides moves LossRate by at most 0.003.

We report LossRate = |lost|/(|retained| + |lifted| + |lost|), aggregated as the post-stratified mean of per-user rates (paired differences are unweighted means).

4.2 Data

We work from the production build output of a large-scale Korean map service, which writes one row per (user, note) for every user whose behaviour changed that day. The service processes 99,993–99,999 users daily (a top-100K cap) and 1,977,833 distinct users over the 36-day window we retain.

Stratification. Elevation can only promote what siblings share, so we stratify by *headroom*, the saving Eq. (3) predicts of a lossless elevation: $\sum_{i \in I}(k_i - 1)$ over a user's tree. Over one full day of production memory it is small, and 17.6% of users sit at zero, so uniform sampling would over-weight users on whom elevation can do almost nothing. Four strata follow: **zero** (0), **low** (1–3), **mid** (4–9), **high** (≥ 10). The **zero** stratum is the control behind this paper's sharpest result: these users have siblings but no shared keyword, so a correct elevation leaves them untouched.

Frozen datasets. Two snapshots, each 500 users per split with 125 per stratum, fixed seed, pinned by content hash. Mech (2026-08-17, 7,924 notes) carries the loss measurements; Eval (2026-07-15, 7,275 notes) adds a held-out MCQ target (Section 4.9). Both reserve a disjoint test half of equal size and stratum balance. Loss numbers in Sections 4–4.7 and the accuracy comparison of Section 4.9 are on the development half; the two experiments on 977 users (Sections 4.5 and 4.8) pool both halves, a pre-registered design that is not a fresh sample. The test half was opened once for the comparisons declared in advance. One qualification applies: after that opening we audited our instrument, corrected two defects (the prose and relocation clauses of Section 4), and recomputed *both* halves under the corrected rule. The correction was motivated by the mechanics of merging, not by anything in the held-out results, and replays stored snapshots deterministically — but it is a post-hoc change, and

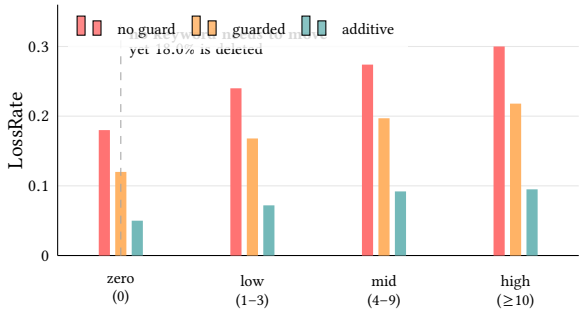


Figure 2: Loss rises with headroom — the saving Eq. (3) predicts a lossless elevation would achieve — for every variant (`gemma-4-31b`, 500 production trees, 125 users per stratum). The leftmost group is the control: these users' siblings share no keyword, so nothing needs to move, and 18.0% is deleted anyway. The guard's benefit grows with headroom too, −0.059 at zero to −0.081 at high. ADDITIVE never rewrites children, so its loss is confined to Level-2 notes acting as parents and it is shown for shape rather than as a like-for-like comparison.

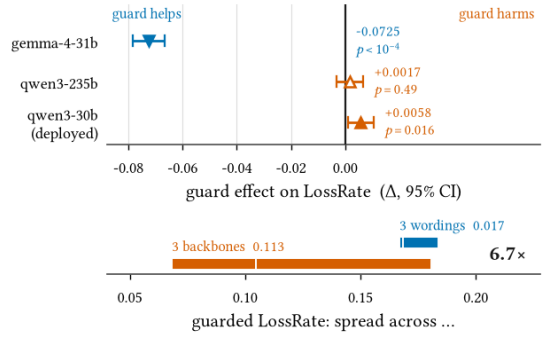


Figure 3: The guard's effect on LossRate by backbone (Δ, 95% CI; blue helps, orange harms). Below, the spread of guarded LossRate across the three backbones (0.113) against the spread across three wordings of the prompt on gemma (0.017), a factor of 6.7.

we name it rather than let "opened once" carry more weight than it can. Because strata are over-sampled relative to the population, every aggregate we report is post-stratified back to the population proportions recorded in the snapshot manifest.

Backbones and isolation. `gemma-4-31b-it` unless stated; `qwen3-30b-a3b-instruct` (deployed) and `qwen3-235b-a22b-instruct` [10] for the model comparison, all served in-house through the same vLLM [11] gateway. The client exposes no seed, so repeated runs measure sampling variance rather than reproducing a draw; three repeats spread ±0.003, a thirtieth of the headline effects below, so single runs are reported elsewhere. Elevation is run alone unless stated: merging renames paths, which breaks keyword tracking, and pruning cannot fire at these node ages (8–34 days against a 120-day threshold). Section 4.6 adds merging back deliberately. Every variant starts from a deep copy of the same tree, so comparisons are paired per user, and the runner aborts if the arms' pre-consolidation keyword totals ever differ.

4.3 Consolidation deletes, including where there is nothing to compress

Figure 2 reports the instrument on 500 production trees. Weighted to the population, unguarded elevation loses 0.253 of all child keywords under the conservative definition, the deployed guard 0.181 ($\Delta = -0.071$). Two features matter more than that headline.

The zero stratum loses 18.0%. These users' siblings share no keyword, so Eq. (3) predicts a keyword-level saving of exactly zero. Nearly a fifth of their concrete tokens disappear anyway. We are careful about what that shows: the agent reads whole summaries, not keyword lists, and zero-stratum siblings still share summary vocabulary (9.1 tokens per parent group against 19.6 in the high stratum), so promoting a shared *theme* is legitimate and the pass need not have been a no-op. The narrower claim still damages: a faithful rewrite promoting a shared theme has no reason to delete a child's *unique* keywords, and those are exactly what disappears. The loss looks collateral to the rewrite rather than a product of compression. The control removes lexical headroom only and we have no rewrite-only baseline, so this names the likeliest cause rather than isolating it.

Loss scales with headroom, and lifting is rare. Loss rises monotonically with the amount genuinely shared (0.180 → 0.300), and so does the guard's benefit (−0.059 → −0.081) Meanwhile keywords classified as *lifted* account for at most 6.2% of the total in any variant. Elevation as executed is overwhelmingly a delete rather than a move: the parent receives an abstraction and the child's concrete token is discarded, not relocated.

Loss versus compression. Between the two variants that rewrite children (summary text −39.4% destructive, −5.8% additive), guarded elevation retains 84% of destructive's compression while cutting loss by 28%.

4.4 The magnitude belongs to the backbone

Identical prompts over identical trees on three backbones: the operation does not behave the same way twice (Figure 3). Gemma compresses hard and loses hard (LossRate 0.255 unguarded in these runs, keywords −34.0%); the Qwen models lose far less (0.102 on the 235B, 0.063 on the 30B), the 235B barely compresses (−11.9%) and the deployed 30B *inflates* (+5.0%) — it returns more keywords than it was given. The guard follows suit: decisive on Gemma ($\Delta = -0.073$, $d = -0.70$; −0.071 in the main runs), inert on the 235B (+0.002), and on the 30B a *significant increase* in loss (+0.006, 95% CI [+0.001, +0.011], $p = 0.016$). The effect is small; its sign is the point. A mitigation validated on one backbone can be harmful on another, while the failure mode itself is universal in direction, since all three models lose keywords in the zero stratum.

The backbone, not the phrasing. Those runs share one wording. Two paraphrases of the elevation instruction — every proposition preserved, only voice, clause order, list format and terminology changed — on gemma over 150 users move guarded LossRate across



Figure 4: What unguarded elevation removes, by token type (`gemma-4-31b`, 500 production trees): share of all lost keywords divided by share of surviving keywords. Above 1 a type is over-represented among losses. Place names and brands, which the deployed guard protects, survive above their base rate; behavioural evidence (most often *repeated clicks*, 184 occurrences) is lost at 2.36× its base rate.

0.167–0.184: a spread of 0.017 against 0.113 across the backbones (0.182, 0.104, 0.069), a factor of 6.7. The guard survives rephrasing, cutting loss by 0.082, 0.068 and 0.054 under the three wordings, each at $p < 10^{-4}$. Phrasing is not nothing (one paraphrase costs +0.015, the other is indistinguishable) but is several times smaller than the choice of model.

4.5 The deployed guard protects the wrong tokens

The guard shipped in production names place names, store names and brands. Typing every keyword and comparing its share of losses against its share of survivors (Figure 4) asks whether those are what disappears. They are not. Place names (ratio 0.58) and brands (0.76) are *under*-represented among losses, so the deployed guard is defending tokens that were never in danger *here* — Section 4.8 finds them destroyed one stage later instead. What is lost in bulk is category vocabulary (*grilled meat*, *gukbap*, *Chinese*, *Korean*) and, at 2.36× its base rate, the behavioural evidence that grounds a claim (*repeated clicks*, 184 occurrences). That second group is the more damaging: the extraction prompt requires every persona claim to cite click evidence, and elevation removes those citations at over twice their base rate, leaving assertions whose support is silently gone.

A guard derived from the measurement. We rewrote the instruction around what the data says is at risk — category, cuisine and business-type nouns, place and brand names, behavioural evidence — disallowing generic superclasses such as "diverse food". The structural operation is unchanged; the cost is one prompt line (wording in Appendix G). LossRate falls from 0.253 to 0.181 (deployed) to 0.160 (v2) (Table 1), all three paired differences surviving Holm correction at $p < 0.0001$; v2 cuts a further 11% of the deployed guard's loss while compressing slightly less — the expected direction, since the tokens it saves are tokens that would otherwise have been deleted.

We tried to price the difference, and could not. Everything above is observational — it says which tokens leave, not what their leaving costs. So we deleted tokens ourselves, which no prompt can make an agent do selectively. On 977 users we built five versions of each real tree: untouched, and four with tokens removed at random, from what the guard protects, from what it leaves exposed, and the entire exposed pool. The three middle arms delete an *identical* count per user, so a difference between them is attributable to which tokens went rather than how many. The design was registered in advance.

Nothing moved, and the exact zero turned out to be structural. The registered contrast was 0.0000 (95% CI $[-0.021, +0.021]$) because the persona is generated from the notes' `summary` text alone — we verified against the deployed agent that `keywords` never reaches the generator, so a deletion confined to that field cannot reach the artifact the probe consumes. Re-running it striking the same victims from the surrounding prose, as real consolidation does, does reach: no two arms share a persona and 26.4% of users change their answer, for a median 367 characters removed. Accuracy still does not move (+0.011, 95% CI $[-0.015, +0.038]$, $p = 0.43$). That is a fact about the probe rather than the memory (Section 4.9).

4.6 Merging relocates; it does not delete

Running the full backward pass isolates what merging adds — and exposes an artifact of the kind an instrument of this shape invites (Table 2). Merging is the only backward operation that *moves* a note: it deletes its sources and writes a unified note under a different path, so an instrument looking only at a keyword's own path and its parent scores everything a merge carried as deleted, and the full pass appears to make merging responsible for 23% and 33% of the total. It is not: of the keywords that rule calls lost, 38.6% unguarded and 48.6% guarded are still present elsewhere in the same tree. Under the relocation-aware scope of Section 4 merging adds almost nothing, while still removing 5.2% of notes, its intended effect. **Merging relocates; it does not delete.** Consolidation loss belongs to elevation, which says where a guard is worth building. It also generalizes: any keyword-level tracker will mis-attribute deletion to whichever operation renames things, here by an order of magnitude.

4.7 The loss reaches what the ranker reads

A downstream ranker never sees the memory; it sees a persona synthesized from a few selected paths, so loss inside the tree matters only if it survives that step. We generated a persona from each tree before and after consolidation and asked how many of the removed keywords also vanished from it ($n = 200$). Propagation is 0.765 without a guard, 0.711 with the deployed one, and 0.713 with v2 — above 70% in every arm, and flat across the two guarded ones. The guard does not make the persona more robust to loss; it reduces how much is lost in the first place, which carries through to 21% fewer keywords missing from the final persona (372 → 295).

The paths do not move. Path selection is itself an LLM call, so a keyword could vanish because a different subtree was read. Recording the chosen paths rules that out: mean Jaccard between pre- and post-consolidation path sets is 0.90–0.92, the sets are *identical* for 67.5%–72.5% of users, and restricting to those users moves propagation by at most 0.021. We have no propagation rate for

Table 1: Guard wording on the development half (500 users, `gemma-4-31b`) and the reserved half. Paired differences are unweighted per-user means with 95% bootstrap CIs; d is Cohen's d; LossRate is post-stratified.

	development half				reserved half		
	LossRate	summary text	Δ vs. no guard [95% CI]	d	LossRate	Δ vs. no guard	d
no guard	0.253	−39.2%	—	—	0.251	—	—
deployed guard	0.181	−32.9%	−0.071 [−0.077, −0.066]	−0.68	0.178	−0.071	−0.68
GUARD-V2	0.160	−29.2%	−0.092 [−0.098, −0.086]	−0.88	0.160	−0.089	−0.86
v2 vs. deployed	—	—	−0.021 [−0.024, −0.016]	−0.22	—	−0.018 [−0.022, −0.014]	−0.19

Table 2: What merging adds, by instrument (`gemma-4-31b`, 500 users). Elevation alone is the baseline; the same-path rule counts a moved keyword as lost.

	unguarded	guarded
elevation alone, LossRate	0.253	0.181
full pass, same-path rule	0.328 (+0.075)	0.268 (+0.088)
share of that total	23%	33%
full pass, relocation-aware	+0.001 (p = 0.75)	+0.007 (p = 0.003)
95% CI	—	[+0.002, +0.012]
lost keywords still in the tree	38.6%	48.6%

Table 3: What the model is given and the MCQ accuracy it reaches (977 users, both halves pooled; the floor is on 471 development-half users). Gains are paired differences with 95% CIs; all survive Holm at $p < 10^{-4}$.

arm	given to the model	accuracy	gain over deployed
no memory	empty persona	0.226	—
deployed	select paths, synthesise	0.459	—
flat + synthesis	all notes, flattened, synthesised	0.575	+0.117 [+0.083, +0.149]
raw notes	all summaries, no synthesis	0.649	+0.190 [+0.156, +0.226]

retained keywords, and Section 4.8 finds the persona discarding much that the tree keeps, so this shows that the loss is not recovered downstream, not that consolidation causes it.

4.8 The delivery path costs more than consolidation does

The tree is not what the ranker reads: between the two sits a delivery path — select a few paths, synthesise a persona — which is itself two compression steps we had never measured. We measured them on the same 977 users and held-out questions, varying only what the model is given (Table 3); instruction, question and option ordering are byte-identical across arms.

Both steps are negative: flattening the tree is worth +0.117, and dropping synthesis as well a further +0.074 ([+0.047, +0.100]). On the 471 users for whom the floor was measured, the deployed persona earns +0.248 over it and the delivery path forgoes +0.184 more: 74% of the deployed memory's own gain, 43% of what the raw notes would earn.

Where it goes. Place names survive consolidation well (Section 4.5) but not delivery: the median user's notes carry six distinct place names and the median retention into the persona is 0.00; 63% of users keep none. The guard protects tokens through the tree and the next stage discards them anyway.

One confound, measured. Distractors are matched on category and review count but not location, so geography is a cue available to the raw-notes arm and not the deployed one. Splitting on whether any option title carries an area name from the user's notes, the gap is +0.285 where it does (n = 172) and +0.170 where it does not (n = 805), both at $p < 10^{-4}$. The confound inflates the effect; it does not produce it.

4.9 Downstream accuracy: ordered but not resolved

Held-out task. To avoid leakage we take an older snapshot (2026-07-15) and hold out everything after: the target is each user's first qualified review in 2026-07-16..08-20, against three same-category distractors matched on review count. Appendix D gives the construction and the filtering it forced.

Accuracy orders exactly inversely to LossRate — 0.452 destructive, 0.466 guarded, 0.472 v2, 0.484 with no consolidation — but no pairwise difference against the no-guard arm is significant (p = 0.46, 0.30, 0.13; widest CI [−0.008, +0.072]), and per-user correlation between LossRate and answering correctly is ≈ 0 in every arm ($|r| \leq 0.05$).

A null is only interpretable against a floor: a NO MEMORY arm scores 0.226, no better than guessing, and the whole consolidation span is an eighth of what memory is worth, so sample size is the obvious explanation. Two results rule it out. Section 4.5's intervention deleted enough to change 26.4% of the answers and left accuracy untouched (+0.011, p = 0.43); Section 4.8 moves the same probe by +0.190 by changing only how the memory is delivered. The probe's resolution is sharply shaped rather than absent: it reads *how* a memory reaches the model and not *which tokens* the tree holds. What the data do support is narrower: guarded consolidation is indistinguishable from not consolidating (Δ = 0.018, p = 0.39) while removing a third of the memory's text.

4.10 Held-out confirmation

Every loss comparison above was developed on one half of each snapshot; the reserved half was opened once, with model, prompts and analysis fixed, then recomputed once when we corrected the instrument (Section 4.2). This matters most for GUARD-V2, whose wording came from the development half's loss profile.

The loss measurements replicate to within 0.003 (Table 1, right), all three paired differences remain significant at $p < 0.0001$ after

Holm correction, and the effect sizes are essentially unchanged. The zero-headroom control holds, at 18.2% against 16.9% (18.0% in Figure 2) — the widest gap, and the one place the reserved half is kinder. GUARD-V2 retains its advantage over the deployed guard, slightly attenuated (−0.021 to −0.018) — the direction expected when a rule tuned on one sample meets another.

The accuracy null replicates, instructively: held-out accuracy runs 0.468, 0.466, 0.448, 0.436 with nothing significant, and the ordering that matched LossRate exactly on the development half breaks. The loss is real and reproducible; this task cannot see it.

5 Deployment

Scale and cost. Forward aggregation costs about ten LLM calls per user per update (Eq. (1), $N \approx 8.0$) and elevation adds 2.99. The dominant cost is implicit extraction at 73K input tokens per user; clustering similar inputs before the LLM [28] and distilling into a smaller model [12] are the plausible reductions.

Consolidation is not free, and is not running. Over 497 consecutive update pairs from 429 users (median gap 17 days), notes per user grow 16.4%, keywords 29.5% and summary text 24.0% per update, with 472/497 pairs increasing. Our deployment has run consolidation once, over 35 users in a test namespace: `last_cleaned_at` is null for every user in current production output. Those growth figures therefore describe the unconsolidated regime rather than a counterfactual.

Sensitive information. Three mechanisms, none described in our earlier writeup. At routing time, paths whose Level-2 segment matches a blocked category (medical, health, hospital, religion, politics) or that contain a blocked token anywhere are dropped before any note is written — why the deployed taxonomy has ten domains rather than the eleven specified. A filtering job separately runs LLM-based detection over every summary, masks the spans it flags, and records a boolean audit column. One inconsistency remains: the implicit-extraction prompt still lists a medical domain, so such content is extracted and only then discarded at routing. The third mechanism deleted a field rather than filtering one: a security review conditioned the memory API's clearance on removing `user_info`, which was withdrawn from the API response, then the prompts and pipeline schema, then the stored Iceberg column and serving index. An external review can retire a field after deployment, and a consolidation policy that assumes a stable schema will not age well.

What the code enforces. Depth ≤ 3 and root-exclusive demographics are requested in prompts and not checked in code — though zero Level-4 paths appeared across all 24 elevation runs and both full-backward runs, so the prompt-level bound holds in practice. Merging takes the maximum update count among its sources rather than the sum, which keeps repeated consolidation a fixed point.

What we can and cannot release. The memory trees are production user data and cannot leave the company, so these measurements are not externally reproducible in the strict sense. The instrument is portable: a few hundred lines with no dependency on our data, applicable to any memory whose notes carry an explicit term field. The agent under measurement reaches us as compiled bytecode, so merge destinations are measured, not argued from source. Per-run artifacts are pinned by content hash, and the mismatches our own audit surfaced are corrected in this version.

6 Conclusion and Future Work

Hierarchical memory systems consolidate themselves, and the step has until now been assessed only by what it saves and what survives downstream. Following individual keywords through it on deployed memory shows it is not doing what its formalization says. A quarter of child keywords vanish from the note and its prose alike, deletion far outnumbers relocation, and users whose siblings share no keyword — for whom nothing needs to move — still lose 18% of their concrete tokens; over 70% of removed keywords are also missing from the persona the ranker reads.

Three consequences follow. The deployed guard protected place names and brands, never at risk, while category vocabulary and the behavioural evidence behind each claim (2.4× its base rate) disappeared; a guard rewritten from the measured profile removes a further 11% of the deployed guard's loss for one prompt line. Sibling merging relocates notes rather than deleting them, and an instrument that does not follow the relocation reports that as roughly a quarter of all loss. And none of it transfers between backbones: a prompt validated on one model is evidence about that model only, and can be worse than nothing elsewhere.

We could not close the chain to accuracy, and we now know why. Deleting the exposed pool from the notes *and* from the prose the persona is written from changes 26.4% of the answers and still leaves accuracy untouched (+0.011, $p = 0.43$). A four-way choice over same-category venues is insensitive to keyword-level memory content, so more users would not have rescued it. The same probe is not insensitive to everything. Holding the memory fixed and varying only its delivery — selecting paths, then synthesizing a persona — costs +0.190 accuracy against handing over the raw notes, three quarters of the gain the deployed memory delivers over an empty persona. The step we instrumented is the cheaper of the two compressions in the pipeline, by roughly a factor of six. Three limitations remain: consolidation has run once over 35 users, so our growth rates describe the unconsolidated regime rather than a controlled comparison; guard-v2 was derived and tested on one backbone; and the prompts and taxonomy are Korean — the failure mode is unlikely to be language-specific, but the guard's wording certainly is.

What we would ask of anyone building such a system is the thing we did not do while operating ours: look at the maintenance step directly. Instrumenting it needs no labels and no downstream task, and a few hundred lines of it found a guard defending tokens that were never at risk, an operation we had blamed for roughly a quarter of the loss that does not delete at all, and a prompt that helps one backbone while harming the one we shipped. And the one thing that did move accuracy was the step we never thought to question.

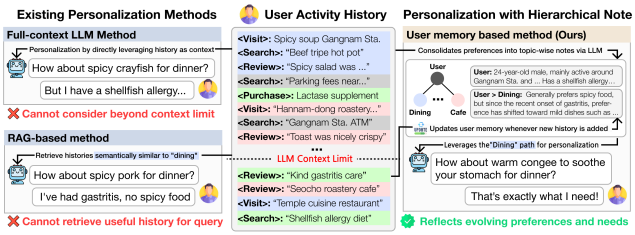


Figure 5: Why the memory has to contract, and what it risks. An illustrative user's preferences have evolved (spicy → mild after gastritis; the deployed system filters health content, Section 5). *Left:* a full-context LLM cannot reason past its context limit, and RAG retrieves entries similar to "dining" while missing the gastritis signal, so both recommend spicy dishes. *Middle:* the raw history exceeds the context window and the decisive evidence is scattered across sessions — the pressure that motivates a compact, structured memory. *Right:* hierarchical notes consolidate that history and surface the evolved preference. The consolidation step this figure takes for granted is what the rest of the paper measures: the same operation that keeps "mild, gastritis" usable can also delete it.

A Motivating example

B Memory schema detail

Each node in the hierarchy is associated with a *Note*, a structured record holding the accumulated preference knowledge for that category:

- **path**: the full hierarchical path (e.g., "User > Dining > Korean Food"),
- **summary**: a natural-language paragraph summarizing the user's preferences and behavioral patterns for this category, integrating both explicit and implicit signals (e.g., for the Korean Food node: "favors spicy and savory dishes such as spicy pork," inferred from repeated visits and supportive review excerpts),
- **keywords**: BM25-oriented keyword tags for retrieval,
- **updated_count**: the number of times the note content has been substantively modified,
- **recently_updated**: the date of the most recent content update (e.g., the Korean Food note advances from 2025/02/19 to 2025/02/25 after new activity arrives, while the Tennis note, last updated on 2024/10/23, remains unchanged).

A sixth field, **user_info**, held demographic and factual attributes at the root (e.g., "24-year-old male, lactose intolerant, frequently active near Gangnam Station"). Root-exclusivity was requested in the routing prompt but, like depth, was never enforced in code. The field was removed from the API response, then from the pipeline, then from the stored data during the period covered by this paper; Section 5 gives the reason. The snapshots our measurements run on predate the removal, and 6.5% of their notes carry a non-empty value.

All notes for a user are stored atomically in a single Parquet file, enabling efficient bulk I/O. Tree structure metadata (parent–child relationships, node levels) is kept in a separate hierarchy file, so the Memory Agent can load, modify, and persist the entire memory state in one read-modify-write cycle per operation.

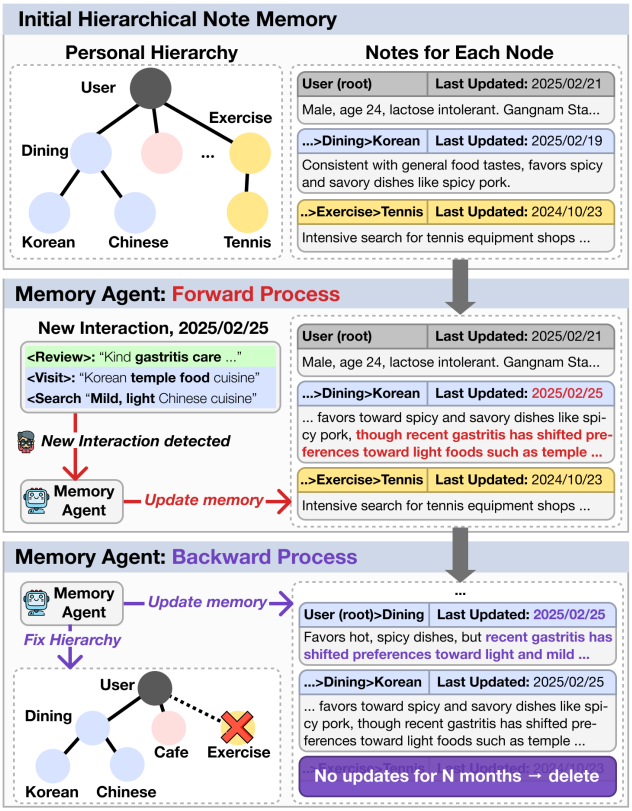


Figure 6: Memory Agent operation. *Initial*: per-node notes anchored to a personal hierarchy (the root demographic note shown here has since been removed). *Forward*: a new interaction selectively updates only the affected note (Korean). *Backward*: a shared trait is elevated from child to parent while concrete subcategory words stay at the child, and stale leaves are pruned. This is the *intended* backward behaviour; Section 4 measures how often the deployed system achieves it.

C Stale leaf pruning

A leaf node n is pruned when it satisfies both conditions simultaneously:

$$\text{updates}(n) \le \tau_u \quad \text{and} \quad \text{age}(n) > \tau_d, \tag{4}$$

where $\text{updates}(n)$ is the cumulative update count (the schema field `updated_count`), $\text{age}(n)$ is days since its last update, τ_u is the minimum update threshold (default 2), and τ_d the staleness threshold in days (default 120). Nodes are processed in decreasing depth order, so pruning a leaf may expose its parent as a new leaf, which is then evaluated in the same pass. The mechanism keeps one-off exploratory behaviour from accumulating while retaining nodes reinforced by repeated interaction.

D Held-out MCQ construction

The target is each user's first qualified review in 2026-07-16..08-20, strictly after the 2026-07-15 snapshot the memory was built from. Distractors are three same-category POIs matched on review count (999/1000 within ±20%), excluding anything the user has touched, with sensitive categories dropped to match deployed routing.

Target availability, not tree eligibility, is the binding constraint: review availability retains only 44.8% of the partition against tree eligibility's 99.2%. We therefore filter on the target first and balance strata within what survives, and require one review in the window rather than our earlier rule's five, which would have retained 1.5% of this cohort. The distractors are matched on category and review count but not on distance from the user, which Section 4.8 treats as a measured confound rather than a controlled one.

E Signal extraction detail

User preferences for places appear through two different channels: explicit signals written by the user, and implicit signals inferred from behavior. We extract a structured preference profile from each channel separately, then merge them in the hierarchical memory. Separating the two channels suits the POI domain, where implicit behavioral signals are abundant but ambiguous and explicit ones are sparse but precise. A third demographic axis is introduced later, during forward aggregation (Section 3.4), with root-exclusive routing.

Explicit Signal Extraction. Explicit signals originate from user-authored content (primarily place reviews), where preferences are stated directly in natural language (e.g., "I love spicy broth" or "the quiet atmosphere was perfect for working"). We process review histories with an LLM-based extraction pipeline that, for each user, produces a set of structured persona items. Each item consists of four fields: (a) *User_Info*, a single atomic fact about the user's lifestyle or preference distilled from the review evidence; (b) *Rationale*, a chain-of-thought justification grounded in specific review dates, place names, and quoted expressions; (c) *User_Vocabulary*, verbatim words and phrases from the user's original text to preserve linguistic identity; and (d) *Reference*, indices into the source review records.

To handle the wide variance in review volume across users, we employ two few-shot prompt strategies: a *rich* variant with detailed examples for users who have written many reviews, and a *sparse* variant with conservative, observation-oriented examples for users with limited review data. The sparse strategy deliberately avoids over-generalizing from insufficient evidence, instead emphasizing factual descriptions of observed behavior.

Implicit Signal Extraction. Implicit signals are derived from non-verbal behavioral traces: search queries, POI clicks, dwell patterns, and place saves. Unlike reviews, these actions do not directly articulate preferences, but consistent patterns across many interactions can reveal latent interests [24]. We construct a profiling pipeline that transforms raw click logs into structured personas through the following steps:

(1) **Sessionization.** Consecutive interactions involving the same query or the same POI within a 60-minute window are grouped into micro-sessions. Each session records the query text, timestamps, day of week, clicked POI name and category, location, review popularity rank, and click count.
(2) **Demographic context attachment.** Available demographic attributes (age, gender, marital status, household composition, occupation, and children information) are prepended to the session log as structured metadata. These attributes were later routed exclusively to the root node's `user_info` field during hierarchical path mapping (Section 3.4), enforcing a clean separation between demographic facts and behavioral preferences.
(3) **LLM-based persona profiling.** The sessionized log is passed to an LLM with strict constraints: (a) all persona claims must cite specific click evidence; (b) hallucination of unobserved behaviors (e.g., inferring residence from visit locations) is prohibited; (c) stereotypical reasoning based on demographics is forbidden; and (d) the output must decompose preferences into predefined POI domains (dining, cafés, travel, medical, beauty, shopping, etc.).

The output is a domain-partitioned persona string, where each domain section describes the user's inferred preferences with concrete behavioral evidence. To capture temporal dynamics, we apply this extraction over sliding windows (e.g., 21-day periods) across the observation period and aggregate the per-window results into a unified implicit persona per user.

F Forward aggregation detail

When new interaction data arrives (either explicit persona extractions or implicit persona profiles), the Memory Agent executes a three-step forward pass to integrate the information into the existing memory. Each step is mediated by an LLM call; for a user with N distinct target paths identified in Step 2, the total cost is

$$\#\text{LLM calls}(u) \;=\; 2 + N(u), \tag{5}$$

where the constant 2 accounts for the single extraction call (Step 1) and the single path-mapping call (Step 2), and Step 3 contributes one call per target path.

Step 1: Preference Profile Extraction. The raw interaction text, combining both implicit behavioral profiles and explicit review extractions, is analyzed by an LLM to separate three categories of information:

- **User information** ($\mathcal{U}$): objective, slowly-changing facts such as age, occupation, and primary activity region.
- **Explicit preferences** ($\mathcal{P}_e$): stated preferences where the user directly expresses likes, dislikes, or requirements in language (e.g., "I prefer acidic coffee" or "I look for outlets at cafés").
- **Implicit preferences** ($\mathcal{P}_i$): latent preferences inferred from non-verbal behavioral patterns (e.g., "tends toward isolated, productivity-oriented spaces" derived from repeated clicks on quiet study cafés).

This three-axis separation isolates demographic facts into a dedicated channel. The LLM is instructed to preserve concrete keywords from the source data rather than abstracting them (e.g., extracting

specific dish names rather than "diverse menu"), to merge semantically similar items into compact statements, and to strictly distinguish between exploratory actions (clicks, searches) and confirmed actions (visits, reservations) to avoid inflating behavioral claims.

Step 2: Hierarchical Path Mapping. Unlike learned taxonomy expansion such as TaxoExpan [56], our routing is performed in-context by an LLM that jointly considers base-taxonomy paths and any user-specific paths created in earlier sessions. The mapping enforces five principles: *(i) root exclusivity for demographics*—while the field existed, $\mathcal{U}$ items were mapped only to the root `user_info` field, never as subcategory nodes; *(ii) context-aware routing*—"prefers quiet atmosphere" goes to "User > Café" if surrounding evidence is café-related, otherwise to the root; *(iii) semantic reuse*—existing paths are preferred when they subsume the new item (a "latte" preference maps to "Coffee/Tea" rather than a new "Latte" sibling); *(iv) hierarchical consistency*—new Level-3 nodes must match the abstraction level of their siblings ("Korean"/"Japanese" under "Dining" yes; "Naengmyeon" no, it belongs under "Korean"), and nodes at Level 4 or deeper are prohibited; *(v) aggregation*—items targeting the same path are grouped into one update call.

Step 3: Note Update. For each target path, an LLM acts as an *editor* rather than appender [14]: it compares new data with existing content, resolves conflicts in favor of recency, removes outdated entries, and consolidates redundant statements. The root node managed demographics in its `user_info` field (since removed; Section 5); sub-nodes are restricted to category-specific preferences. Substantive changes increment `updated_count` and refresh `recently_updated`.

G Guard wording

The prompts are Korean; the rendering below is ours. The deployed guard is one instruction appended to the elevation prompt: proper nouns — place names, store names, brands — are kept verbatim at the child and never lifted to the parent. The wording derived from the measured loss profile (guard-v2) replaces it with:

> **Preserve concrete nouns.** Never abstract or delete the following; leave them verbatim in the child note: (1) names of categories, business types and kinds of food (e.g., Korean, Chinese, Japanese, gukbap, fried chicken, jokbal, sushi, izakaya, bakery, café, grilled meat, raw fish); (2) place, store and brand names; (3) behavioural-evidence phrases (e.g., repeated clicks, repeated searches, sustained interest, saved POIs). Do not replace them with umbrella terms such as "diverse food" or "dining preference". State at the parent only the tendency the children share, and do not move a child's concrete nouns to the parent.

References

[1] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald. On faithfulness and factuality in abstractive summarization. In *ACL*, 2020.

[2] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 2025. arXiv:2311.05232.

[3] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *EMNLP*, 2023. arXiv:2310.05736.

[4] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr. Quantifying language models' sensitivity to spurious features in prompt design, or: How I learned to start worrying about prompt formatting. In *ICLR*, 2024. arXiv:2310.11324.

[5] M. Mizrahi, G. Kaplan, D. Malkin, R. Dror, D. Shahaf, and G. Stanovsky. State of what art? A call for multi-prompt LLM evaluation. *Transactions of the ACL*, 12, 2024.

[6] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, and J.-R. Wen. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024.

[7] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *NeurIPS Datasets and Benchmarks Track*, 2023.

[8] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, S. Wang, K. Zhang, Y. Wang, W. Gao, L. Ni, and J. Guo. A survey on LLM-as-a-Judge. *arXiv preprint arXiv:2411.15594*, 2025.

[9] F. Fabbri, G. Penha, E. D'Amico, A. Wang, M. De Nadai, J. Doremus, P. Gigioli, A. Damianou, O. Stål, and M. Lalmas. Evaluating podcast recommendations with profile-aware LLM-as-a-Judge. In *RecSys*, 2025. arXiv:2508.08777.

[10] A. Yang, A. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, G. Dong, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[11] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.

[12] B. Jiang, Y. Yuan, M. Shen, Z. Hao, Z. Xu, Z. Chen, Z. Liu, A. R. Vijjini, J. He, H. Yu, R. Poovendran, G. Wornell, L. Ungar, D. Roth, S. Chen, and C. J. Taylor. PersonaMem-v2: Towards personalized intelligence via learning implicit user personas and agentic memory. *arXiv preprint arXiv:2512.06688*, 2025.

[13] B. Jiang, Z. Hao, Y.-M. Cho, B. Li, Y. Yuan, S. Chen, L. Ungar, C. J. Taylor, and D. Roth. Know me, respond to me: Benchmarking LLMs for dynamic user profiling and personalized responses at scale. In *COLM*, 2025. arXiv:2504.14225.

[14] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

[15] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang. A-MEM: Agentic memory for LLM agents. In *NeurIPS*, 2025. arXiv:2502.12110.

[16] S. Zhou and J. Han. A simple yet strong baseline for long-term conversational memory of LLM agents. *arXiv preprint arXiv:2511.17208*, 2025.

[17] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[18] N. Pollertlam and W. Kornsuwannawit. Beyond the context window: A cost-performance analysis of fact-based memory vs. long-context LLMs for persistent agents. *arXiv preprint arXiv:2603.04814*, 2026.

[19] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the ACL*, 2024.

[20] T. Ge, X. Chan, X. Wang, D. Yu, H. Mi, and D. Yu. Scaling synthetic data creation with 1,000,000,000 personas. *arXiv preprint arXiv:2406.20094*, 2024.

[21] V. Samuel et al. PersonaGym: Evaluating persona agents and LLMs. In *Findings of EMNLP*, 2025. arXiv:2407.18416.

[22] J. Tan, L. Yang, Z. Liu, Z. Liu, R. Murthy, T. M. Awalgaonkar, J. Zhang, W. Yao, M. Zhu, S. Kokane, S. Savarese, H. Wang, C. Xiong, and S. Heinecke. PersonaBench: Evaluating AI models on understanding personal information through accessing (synthetic) private user data. In *Findings of ACL*, 2025. arXiv:2502.20616.

[23] M. J. Ryan, O. Shaikh, A. Bhagirath, D. Frees, W. Held, and D. Yang. SynthesizeMe! Inducing persona-guided prompts for personalized reward models in LLMs. In *ACL*, 2025. arXiv:2506.05598.

[24] Y. Zhang, Y. Zhang, Y. Wang, T. Chen, and H. Yin. ProEx: A unified framework leveraging large language model with profile extrapolation for recommendation. In *KDD*, 2026. arXiv:2512.00679.

[25] S. Feng, H. Lyu, C. Chen, and Y.-S. Ong. Where to move next: Zero-shot generalization of LLMs for next POI recommendation. *arXiv preprint arXiv:2404.01855*, 2024.

[26] C. Sun, K. Yang, R. G. Reddy, Y. R. Fung, H. P. Chan, K. Small, C. Zhai, and H. Ji. Persona-DB: Efficient large language model personalization for response prediction with collaborative data refinement. *arXiv preprint arXiv:2402.11060*, 2024.

[27] Z. Tan, J. Yan, I.-H. Hsu, R. Han, Z. Wang, L. T. Le, Y. Song, Y. Chen, H. Palangi, G. Lee, A. Iyer, T. Chen, H. Liu, C.-Y. Lee, and T. Pfister. In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents. In *ACL*, 2025. arXiv:2503.08026.

[28] Y. Shi, W. Xu, Z. Zhang, X. Zi, Q. Wu, and M. Xu. PersonaX: A recommendation agent oriented user modeling framework for long behavior sequence. In *Findings of ACL*, 2025. arXiv:2503.02398.

[29] V. Karpukhin et al. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 2020.

[30] G. Izacard et al. Unsupervised dense information retrieval with contrastive learning. *TMLR*, 2022.

[31] P. Grünwald. *The Minimum Description Length Principle.* MIT Press, 2007.

[32] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

[33] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

[34] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *UIST*, 2023.

[35] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.

[36] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *ICLR*, 2024.

[37] W.-C. Kang and J. McAuley. Self-attentive sequential recommendation. In *ICDM*, 2018.

[38] F. Sun, J. Liu, J. Wu, C. Pei, X. Lin, W. Ou, and P. Jiang. BERT4Rec: Sequential recommendation with bidirectional encoder representations from Transformer. In *CIKM*, 2019.

[39] J. Feng, Y. Li, C. Zhang, F. Sun, F. Meng, A. Guo, and D. Jin. DeepMove: Predicting human mobility with attentional recurrent networks. In *WWW*, 2018.

[40] S. Geng, S. Liu, Z. Fu, Y. Ge, and Y. Zhang. Recommendation as language processing (RLP): A unified pretrain, personalized prompt & predict paradigm (P5). In *RecSys*, 2022.

[41] K. Bao, J. Zhang, Y. Zhang, W. Wang, F. Feng, and X. He. TALLRec: An effective and efficient tuning framework to align large language model with recommendation. In *RecSys*, 2023.

[42] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. In *ICLR*, 2021.

[43] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of AAAI*, 2024.

[44] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *NeurIPS*, 2024.

[45] A. Salemi, S. Mysore, M. Bendersky, and H. Zamani. LaMP: When large language models meet personalization. In *ACL*, 2024.

[46] Z. Tan, Q. Zeng, Y. Tian, Z. Liu, B. Yin, and M. Jiang. Democratizing large language models via personalized parameter-efficient fine-tuning (OPPU). In *EMNLP*, 2024.

[47] S. Yang, J. Liu, and K. Zhao. GETNext: Trajectory flow map enhanced transformer for next POI recommendation. In *SIGIR*, 2022.

[48] Y. Luo, Q. Liu, and Z. Liu. STAN: Spatio-temporal attention network for next location recommendation. In *WWW*, 2021.

[49] J. Liao, S. Li, Z. Yang, J. Wu, Y. Yuan, X. Wang, and X. He. LLaRA: Large language-recommendation assistant. In *SIGIR*, 2024.

[50] W. Sun, L. Yan, X. Ma, S. Wang, P. Ren, Z. Chen, D. Yin, and Z. Ren. Is ChatGPT good at search? Investigating large language models as re-ranking agents. In *EMNLP*, 2023.

[51] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg. RULER: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

[52] L. Wu, Z. Zheng, Z. Qiu, H. Wang, H. Gu, T. Shen, C. Qin, C. Zhu, H. Zhu, Q. Liu, H. Xiong, and E. Chen. A survey on large language models for recommendation. *World Wide Web*, 27, Article 60, 2024.

[53] Y. Zhuang, H. Sun, Y. Yu, R. Qiang, Q. Wang, C. Zhang, and B. Dai. HYDRA: Model factorization framework for black-box LLM personalization. In *NeurIPS*, 2024.

[54] Y. Wang, Z. Jiang, Z. Chen, F. Yang, Y. Zhou, E. Cho, X. Fan, Y. Lu, X. Huang, and Y. Yang. RecMind: Large language model powered agent for recommendation. In *NAACL Findings*, 2024.

[55] G. Kamradt. Needle in a haystack—pressure testing LLMs. GitHub repository, `gkamradt/LLMTest_NeedleInAHaystack`, 2023.

[56] J. Shen, Z. Shen, C. Xiong, C. Wang, K. Wang, and J. Han. TaxoExpan: Self-supervised taxonomy expansion with position-enhanced graph neural network. In *WWW*, 2020.